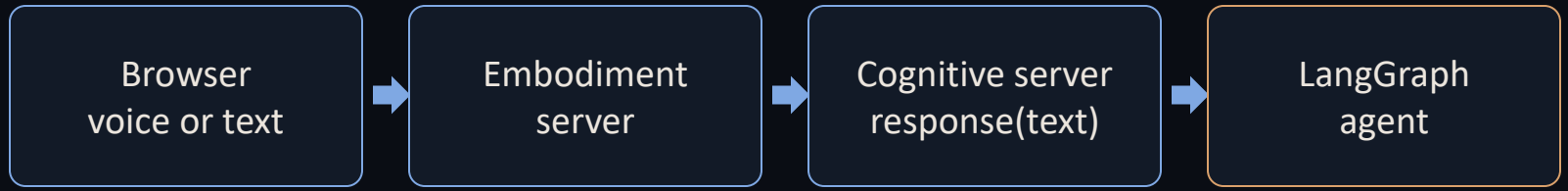
AI-powered Virtual Humans

LLM-Powered Cognitive Architectures

Module 2 - Chapter 03
Integration with the Virtual Human



The Avatar Loop



AgentReply: speech_text, emotion, expression, gesture

The reply travels back: the avatar speaks, moves and changes expression.

What Stays the Same

Frontend

Sends only text.
Plays the reply.
Unchanged.

Contract

response(text)
→ AgentReply

Server routes

/agent
/conversation/clear

Memory is a backend decision: select another agent import.

The Graph Behind response() [1]



InMemorySaver checkpointer · thread_id = "avatar"

respond: prompt template + recent messages → ModelReply

State: Messages and Reply [2]

```
class AvatarState(MessagesState):
    reply: dict
```

messages

The conversation, appended by the reducer.
Only the spoken text is stored.

reply

The latest ModelReply for the avatar: emotion, expression, gesture.

Prompt Templates and Message History [3]

```
prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template(
        "You are a virtual human. Time: {current_time}."),
    MessagesPlaceholder("messages"),
])
```

The system template fills variables. The placeholder keeps message roles.

The respond Node [2]

```
def respond(state: AvatarState):
    answer = structured_client.invoke(
        prompt.format_messages(
            current_time=current_time,
            messages=state["messages"][-MAX_MESSAGES:]))
    return {"messages": [AIMessage(answer.speech_text)],
            "reply": answer.model_dump()}
```

Structured output from Module 1 + rolling buffer from Chapter 02.

The Thread Lives in the Backend [1]

```
THREAD_ID = "avatar"
config = {"configurable": {"thread_id": THREAD_ID}}

def response(text: str) -> AgentReply:
    result = graph.invoke({"messages": [HumanMessage(text)]}, config)
    return AgentReply(**result["reply"])
```

The browser sends only text; every request continues the same thread.

Clear Conversation [1]




```
def clear_conversation():
    checkpointer.delete_thread(THREAD_ID)
```

The next message starts an empty conversation.

Together: Clear Memory through the API

```python
@app.post("/conversation/clear")
def clear_conversation():
    selected_agent = sys.modules[response.__module__]
    clear_agent_memory = getattr(
        selected_agent, "clear_conversation", None)
    if callable(clear_agent_memory):
        clear_agent_memory()
    return {"status": "ok"}
```

Guided exercise: agent hook, server route and browser test

Always Answer

Model error or timeout

A short apology and a shrug.

No structured reply

The same fallback reply.

History in order

Human and AI messages still alternate.

A failed call must not break the avatar or the saved conversation.

Try It on the Avatar

Select the agent

In vh_cognitive_server.py keep one import:
agents.Module_2.Chapter_03
.langgraph_avatar_agent

Test the memory

Tell your name and favourite colour.
Ask for both.
Clear the conversation and ask again.

Save, then restart the cognitive server in the avatar Settings.

Your Turn

Persona

Change the system instruction: a museum guide with its own name.

Buffer

Set MAX_MESSAGES = 4. When does the avatar forget?

Optional

Return a different gesture when greeting and when saying goodbye.

Edit the agent file, save and restart the cognitive server each time.

Module 2 Recap [1] [2]

Chapter 01

Graphs: state, nodes, edges.

Chapter 02

Messages, reducers, checkpointer and threads.

Chapter 03

The avatar remembers the conversation.

Final lab: tools, coding agents and a weather presenter.

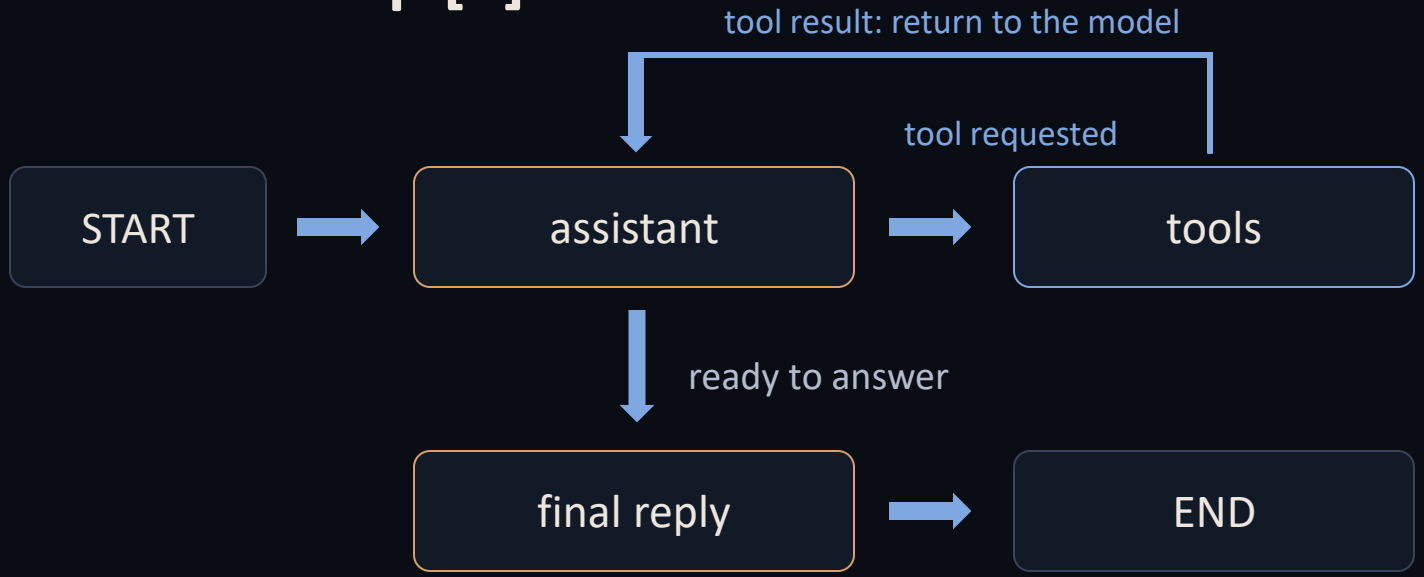
Tools: Actions Outside the Model [4]

A tool lets an agent retrieve data or perform an action.

The model chooses an available tool and supplies its inputs.

The program runs the tool and returns its result to the model.

The Tool Loop [4]



The program returns the tool result to the model, which prepares the reply.

OpenWeather: Request and JSON [7]

```
GET https://api.openweathermap.org/data/2.5/weather
    ?lat=41.90&lon=12.50&appid=YOUR_KEY&units=metric
```

A JSON response contains the observation:

```
{
  "name": "Rome",
  "main": {"temp": 18.0},
  "weather": [{"description": "clear sky"}]
}
```

Get an OpenWeather API Key [10] [11]

1. Create an OpenWeather account

2. Confirm your email using the verification message.

3. Open API keys and copy your personal key (APPID)

Create it before class: automatic activation can take up to two hours.

Save Your API Key

Save the key in a YAML file wherever you prefer.

```yaml
api_key: "YOUR_OPENWEATHER_KEY"
```

Tell Codex the file path so the weather agent can read it.

Coding Agents [5] [6]

Claude Code (Anthropic) and Codex (OpenAI)

They read project files, write code and run it to check the result.

Give a goal, inspect the changes, try the result and ask for improvements.

Codex: Installation and Setup [5]

1. Install the desktop app from the official quickstart

2. Sign in with ChatGPT and open a new folder: codex-demo.

3. Try one small task before opening the course project.

Codex Demo: A Browser Game

Ask Codex in the empty codex-demo folder:

Create a simple, lightweight game in the style of {game style}.
Put HTML, CSS and JavaScript in one index.html file.
It should open directly in a browser.
Add clear controls, a score and a restart button.

Choose a style: Snake, Pong, a platformer or an endless runner.

Exercise 1: A Virtual Meteorologist

Open the course project and start from langgraph_avatar_agent.py.

Use Codex to add current weather while keeping memory and animations.

First version: spoken replies and avatar behaviour. Leave the board unused.

Exercise brief and complete Codex prompt

Prompt: Create the Weather Agent

Create weather_avatar_agent.py from langgraph_avatar_agent.py.
Use OpenWeather for current-weather questions.
Read the key from the YAML file I specify.
Keep memory, animations and Clear conversation.
Speech: spoken words only, no emoji or written actions.
Use chat and message prompt templates.
Return whiteboard=None. Keep the original agent unchanged.
Show how to select and test the new agent.

Ask Codex to explain its plan, then review the diff and run the avatar.

Test the First Weather Agent

Introduce yourself, then ask the avatar to recall your name.

Ask for Rome, Italy. Follow with: What about Milan?

Ask for a happy greeting and check the animation. Clear and retry.

Check an API failure. The first version does not update the board.

How Could We Use the Whiteboard?

The weather presenter works. How could an image help?

Show an image that represents the weather requested.

Write your own Codex prompt to extend the agent.

Whiteboard Playground

Upload an image and apply it to the avatar scene.

Replace it, keep it unchanged, then clear it.

What would our weather agent need to automate?

Exercise 2: Write Your Own Prompt

Ask Codex to show a weather image on the whiteboard.

`Agent: virtual_human_project/vh_cognitive_architecture_server/agents/Module_2/Chapter_03/weather_avatar_agent.py`

`API: virtual_human_project/vh_embodiment_server/README.md`

Explain what already works and include your YAML file path.

References

[1] LangGraph persistence

Checkpoints, threads and deleting a thread

[2] ChatOllama structured output

with_structured_output and its methods

[3] Chat prompt templates

System templates and message placeholders

[4] LangGraph tool loop

Model decisions, tools and tool results

[5] Codex quickstart

Desktop installation and ChatGPT sign-in

[6] Claude Code

Coding agent overview

[7] OpenWeather current weather

Current conditions and metric units

[8] OpenWeather geocoding

City names to coordinates

References (continued)

[9] OpenWeather icons

Weather codes and condition icons

[10] OpenWeather account and keys

Registration, email verification and activation

[11] OpenWeather free access

Current Weather and Geocoding availability